UADE · Universidad Argentina de la Empresa

Proceso de Desarrollo de Software

Docente: Prof. Jonathan Pepe

Informe técnico

Sokoban en Java y Swing

Decisiones de diseño y su justificación

Equipo: [Nombre del equipo]

Integrante	Rol en el proyecto
Cabrera, Martín	Arte y recursos gráficos (`assets/`); integración; funcionalidades adicionales
de Iglesia, Luca	Interfaz gráfica Swing (`sokoban.vista`) y contrato del modelo
Castillo, Pedro	Dominio y personaje (`sokoban.modelo`, `niveles/`)
Santamarina, Lucía	Dominio y personaje (`sokoban.modelo`, `niveles/`)
Coghlan, Mora	Sonido (`sokoban.sonido`, `assets/sonidos/`)

Trabajo integrador · Noviembre de 2026

Repositorio: https://github.com/martincabrera8/sokoban (rama main)

Índice

1. Introducción y alcance **3**

2. Arquitectura general **4**

2.1 Paquetes y responsabilidades 4

2.2 MVC y el DTO EstadoJuego 5

2.3 Composición en Main 5

3. Decisiones de diseño y justificación **6**

3.1 Facade: Juego y JuegoSokoban 6

3.2 Observer: la vista se actualiza sola 7

3.3 Singleton: Imagenes y Sonidos 8

3.4 Strategy: los tres tipos de caja 9

3.5 Mediator: el movimiento 10

3.6 Decorator: el sonido 11

3.7 State: sonido activado o silenciado 12

3.8 Memento en el deshacer 13

3.9 Resumen de principios SOLID y GRASP 14

4. Reglas del juego implementadas **15**

4.1 Formato de los niveles y validación 16

4.2 Tipos de caja 16

4.3 Terreno resbaladizo 16

4.4 Deshacer, victoria e interfaz 16

4.5 Sonidos 17

5. Criterio de puntaje **17**

6. Funcionalidades adicionales **18**

6.1 DeLorean: volver atrás en el tiempo (Memento) 18

6.2 Repetición de la partida (Command) 20

7. Diagramas **21**

7.1 Diagramas de patrones 21

7.2 Diagrama de secuencia: mover y empujar una caja 21

7.3 Diagrama de clases completo 23

8. Limitaciones conocidas y trabajo futuro **24**

9. Cómo ejecutar **24**

10. Declaración de uso de herramientas de IA **25**

Referencias **25**

1. Introducción y alcance

Este informe documenta las decisiones de diseño del trabajo integrador de la materia: una versión del clásico Sokoban desarrollada en Java 17 con Swing. La consigna pone el foco de la evaluación en «la correcta aplicación de los principios y patrones de diseño presentados en la asignatura (SOLID, GRASP, MVC, GOF)», por lo que cada decisión se presenta con el problema concreto que resuelve, las clases reales que participan, la alternativa descartada y los costos que el equipo reconoce.

Alcance. El informe describe el código de la rama `main` del repositorio al 10 de noviembre de 2026: las cuatro partes del equipo integradas (arte, interfaz gráfica, dominio y sonido) y las dos funcionalidades adicionales que exige la consigna, **DeLorean** (viaje en el tiempo, patrón Memento) y **Repetición de la partida** (patrón Command). Todas las reglas obligatorias están implementadas y se probaron en pantalla con los tres niveles incluidos.

Fuentes. Todo lo afirmado sobre el comportamiento del juego surge del código fuente (`src/main/java/sokoban/`), de los archivos de nivel (`niveles/`), del `README.md` y de las notas `docs/dominio.md` y `docs/sonido.md`. Los nombres de clases, métodos y constantes se citan tal como están en el código. Los valores de ejemplo de puntaje se obtuvieron ejecutando el juego.

Organización del trabajo. El equipo se dividió por partes: Martín produjo el arte, integró las partes e implementó las funcionalidades adicionales; Luca hizo la interfaz gráfica y el contrato entre la vista y el dominio; Pedro y Lucía, el dominio (reglas, personaje y niveles), y Mora, el sonido. Esta división fue posible por una decisión de arquitectura: las partes no se conocen entre sí y solo dependen de un contrato de interfaces (sección 2).

Estructura. La sección 2 presenta la arquitectura; la 3, cada patrón con su justificación; la 4 relaciona las reglas de la consigna con el código; la 5 explica el puntaje; la 6 describe las funcionalidades adicionales; la 7 reúne los diagramas; la 8 enumera limitaciones; la 9 explica cómo ejecutar el juego y la 10 declara el uso de inteligencia artificial.

2. Arquitectura general

La aplicación se organiza según el patrón arquitectónico **MVC** (Modelo, Vista, Controlador) y en paquetes con dependencias en un solo sentido. La pieza central es un **contrato** de cuatro tipos en `sokoban.modelo` (`Juego`, `ObservadorJuego`, `EstadoJuego` y `Direccion`): la vista, los comandos, el dominio y el sonido dependen de ese contrato y no entre sí.

2.1 Paquetes y responsabilidades

Paquete o carpeta	Contenido	Autor
`sokoban`	`Main`: punto de entrada y raíz de composición.	Luca (cuatro líneas de Mora)
`sokoban.vista`	`VentanaPrincipal`, `MenuPrincipal`, `PantallaJuego`, `PanelHUD`, `PanelTablero`, `Cartel`, `Imagenes`, `Epoca`.	Luca; `Epoca` y los agregados del DeLorean y la repetición, Martín
`sokoban.comandos`	`Comando`, `ComandoMover`, `ComandoDeshacer`, `ComandoViajarEnElTiempo`, `GrabadoraDePartida`, `Repeticion`.	Martín
`sokoban.modelo` (contrato)	`Juego`, `ObservadorJuego`, `EstadoJuego`, `Direccion`.	Luca
`sokoban.modelo` (dominio) y subpaquetes `entidades`, `estrategias`, `movimiento`	`JuegoSokoban`, `CargadorNivel`, `EstadoNivel`, `Posicion`, `Jugador`, `Caja`, `ComportamientoCaja` y sus tres estrategias, `MediadorMovimiento`, `MotorMovimiento`; `RecuerdoNivel`.	Pedro y Lucía; `RecuerdoNivel` y el plutonio, Martín
`sokoban.modelo.tiempo`	`MaquinaDelTiempo`.	Martín
`sokoban.sonido`	`JuegoConSonido`, `Sonidos`, `EstadoSonido`, `SonidoActivado`, `SonidoSilenciado`, `Sonido`, `SonidosDeInterfaz`.	Mora
`assets/`, `niveles/`	107 imágenes PNG; 16 efectos WAV en `assets/sonidos/`; 3 niveles en texto.	Martín (imágenes), Mora (sonidos), Pedro y Lucía (niveles)

Regla de dependencias. Ningún archivo de `sokoban.modelo` importa algo de `sokoban.vista`, `sokoban.comandos` o `sokoban.sonido`. La vista solo importa tipos del contrato y de `sokoban.comandos`; los comandos y el sonido solo importan tipos del contrato. La única clase que conoce las implementaciones concretas es `Main`. La regla se verifica buscando las sentencias `import` de cada paquete.

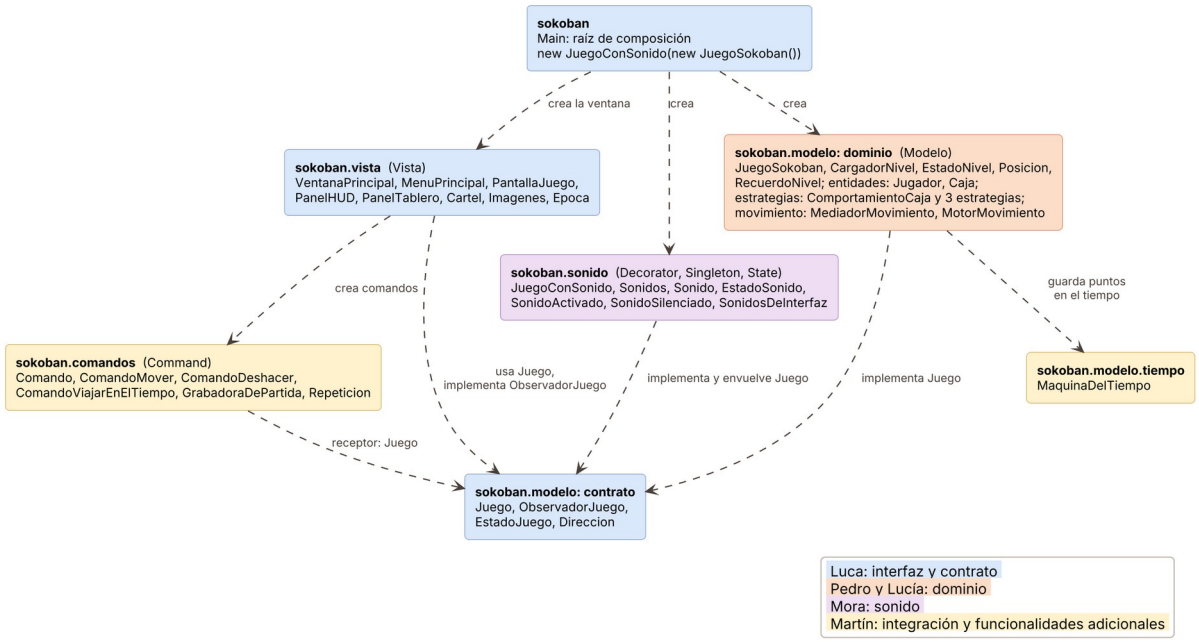


Figura 1. Paquetes y dependencias. Todas las flechas llegan al contrato; el color indica el autor.

2.2 MVC y el DTO EstadoJuego

Modelo: el dominio, con `JuegoSokoban` como punto de entrada. **Vista:** los paneles Swing, que dibujan y nunca deciden reglas. **Controlador:** en Swing es habitual que el controlador de entrada viva en la vista. `PantallaJuego` traduce las flechas, la tecla T y los botones del HUD (que le avisan por *callbacks*) en **comandos** que ejecuta la `GrabadoraDePartida`, y los comandos llaman a `Juego` (sección 6.2). En términos de GRASP, el **Controlador** que recibe los eventos del sistema y coordina el dominio es `JuegoSokoban`.

La vista recibe el estado como un **DTO**: `EstadoJuego` es un `record` inmutable con nivel, movimientos, empujes, deshacer usados, cargas de plutonio, viajes en el tiempo, puntaje, si el nivel está completo y el tablero como lista de líneas de texto (un símbolo por celda, el formato de los archivos de nivel). El constructor compacto copia la lista con `List.copyOf`, de modo que ningún observador puede modificar lo que reciben los demás. La vista nunca recibe un `EstadoNivel`, una `Caja` o el `Jugador`.

Costo reconocido: un carácter por celda no alcanza para dos elementos superpuestos. El dominio genera símbolos derivados (`f`, `G`, `g`, `K`, `c`, `O`), pero quedan casos sin símbolo propio (sección 8).

2.3 Composición en Main

`Main` es la **raíz de composición**: el único lugar que nombra las clases concretas. Crea el dominio, lo envuelve con el sonido y entrega el resultado a la ventana como un `Juego`, todo en el hilo de eventos de Swing (EDT):

```
SwingUtilities.invokeLater(() -> {
    Juego juego = new JuegoConSonido(new JuegoSokoban());
    new VentanaPrincipal(juego).setVisible(true);
    SonidosDeInterfaz.instalar();   // arranque, clic de botones y tecla M
});
```

La dependencia se inyecta por constructor: `VentanaPrincipal` recibe el `Juego` y se lo pasa a `PantallaJuego`. `PanelHUD` ni siquiera conoce a `Juego`: recibe cuatro `Runnable` (deshacer, reiniciar, viajar y pausar). Las pantallas se alternan con un `CardLayout` y se comunican por *callbacks*. Esta inversión de dependencias (DIP) permitió desarrollar la vista con una implementación provisoria del contrato antes de que existiera el dominio, y conectar luego el dominio y el sonido cambiando `Main` y sumando símbolos al mapa de dibujo de `PanelTablero`.

Épocas. El enum `Epoca` (agregado en la integración) asocia a cada nivel un fondo, un piso y una pared (1955, 1985 y 2015, en rotación). Agregar una época es agregar una constante, sin modificar el dibujo (OCP).

3. Decisiones de diseño y justificación

Para cada patrón se indica el problema que resuelve en este proyecto, los participantes con sus clases reales, por qué se prefirió frente a una alternativa, qué principios SOLID y GRASP sostiene y qué costos tiene. Memento y Command, que sostienen las funcionalidades adicionales, se tratan en la sección 6, salvo el uso de Memento en el deshacer (3.8).

Patrón	Tipo	Dónde	Para qué
Facade	Estructural	`Juego` / `JuegoSokoban`; `Sonidos`	Una puerta simple al dominio; esconder `javax.sound`
Observer	Comportamiento	`ObservadorJuego`, `PanelHUD`, `PanelTablero`	Actualizar la vista cuando cambia la partida
Singleton	Creacional	`Imagenes`, `Sonidos`	Una sola caché de imágenes y de sonidos
Strategy	Comportamiento	`ComportamientoCaja` y tres estrategias	Reacción de cada tipo de caja al empuje
Mediator	Comportamiento	`MediadorMovimiento`, `MotorMovimiento`	Reglas de movimiento en un solo lugar
Decorator	Estructural	`JuegoConSonido`	Agregar sonido sin modificar el juego
State	Comportamiento	`EstadoSonido`, `SonidoActivado`, `SonidoSilenciado`	Silenciar y activar con la tecla M
Memento	Comportamiento	`RecuerdoNivel`, `JuegoSokoban`, `MaquinaDelTiempo`	Deshacer y DeLorean
Command	Comportamiento	`Comando`, `GrabadoraDePartida`, `Repeticion`	Grabar y repetir la partida

3.1 Facade: Juego y JuegoSokoban

Problema. Mover al jugador implica guardar un recuerdo para deshacer, consultar el tablero, empujar la caja, aplicar el efecto de su tipo, dejarla deslizar en el hielo, recoger plutonio, guardar puntos en el tiempo, contar movimientos y empujes y avisar a la vista. La vista no debería conocer ese subsistema; solo necesita pedir `mover(Direccion.DERECHA)`.

Rol	Clases del proyecto
Fachada	`Juego` (interfaz): `getCantidadNiveles`, `iniciarNivel`, `mover`, `deshacer`, `reiniciar`, `viajarEnElTiempo`, `getEstado`, `agregarObservador`
Fachada concreta	`JuegoSokoban`
Subsistema	`CargadorNivel`, `EstadoNivel`, `MotorMovimiento`, `Jugador`, `Caja` y sus estrategias, `Posicion`, `RecuerdoNivel`, `MaquinaDelTiempo`
Clientes	`VentanaPrincipal`, `PantallaJuego` y los comandos (`ComandoMover`, `ComandoDeshacer`, `ComandoViajarEnElTiempo`)

Por qué y no la alternativa. Si la vista manipulara `EstadoNivel` o `Jugador`, quedaría atada a la representación interna y podría saltear reglas. La fachada se definió como **interfaz** y no como clase concreta, como suele aparecer en el GoF: así se combina Facade con DIP, lo que permitió desarrollar la vista contra una implementación provisoria e interponer después el decorador de sonido sin que la vista lo note.

Principios. **DIP**: los clientes dependen de la abstracción `Juego`; `Main` inyecta la implementación. **LSP** y **OCP**: `JuegoSokoban` y `JuegoConSonido` son intercambiables detrás de `Juego`. GRASP: **Controlador** (`JuegoSokoban` recibe los eventos del sistema), **Indirección**, **Variaciones protegidas** y **Bajo acoplamiento**.

Costos y límites. `Juego` mezcla las operaciones del juego con `agregarObservador`; por ISP podría dividirse. Sumar el DeLorean obligó a agregar `viajarEnElTiempo` a la interfaz y, por lo tanto, a sus dos implementaciones: es el precio de una fachada única. `JuegoSokoban`, además de coordinar, conserva el historial, los contadores y el puntaje; es el primer candidato a dividir si crece. La obligación de notificar después de cada cambio está en el Javadoc de `Juego`, no garantizada por el tipo.

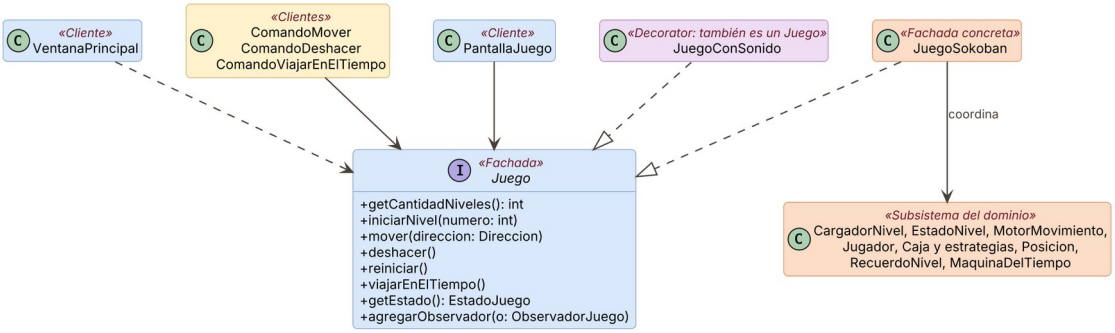


Figura 2. Facade: los clientes usan solo Juego; JuegoSokoban coordina el subsistema del dominio.

3.2 Observer: la vista se actualiza sola

Problema. Después de cada cambio hay que redibujar el HUD y el tablero. Si el dominio llamara directamente a `PanelHUD` y `PanelTablero`, dependería de la vista y de Swing.

Rol	Clases del proyecto
Sujeto	`Juego` (`agregarObservador`)
Sujeto concreto	`JuegoSokoban`: guarda una `List<ObservadorJuego>` y la recorre en `notificar()` al final de `iniciarNivel`, `mover`, `deshacer` y `viajarEnElTiempo` (`reiniciar` llama a `iniciarNivel`)
Observador	`ObservadorJuego.actualizar(EstadoJuego)`
Observadores concretos	`PanelHUD` (indicadores; habilita el botón del DeLorean solo si hay cargas) y `PanelTablero` (dibujo; orienta al personaje según el paso entre dos estados, lo que sirve también en la repetición); los registra `PantallaJuego`
Estado que viaja	`EstadoJuego` (modelo *push*: una sola instantánea inmutable para todos)

Por qué y no la alternativa. Que el dominio invoque a los paneles invierte la dependencia entre capas. Que la vista consulte el estado con un temporizador desperdicia trabajo y puede mostrar estados intermedios. Con Observer, el sujeto solo conoce la interfaz y la vista se entera exactamente cuando hay un cambio.

Principios. **DIP** (el sujeto depende de `ObservadorJuego`), **OCP** (un observador nuevo no modifica el sujeto), **ISP** (un único método). GRASP: **Polimorfismo**, **Bajo acoplamiento**, **Variaciones protegidas**.

Costos y límites. No existe `quitarObservador`; no hace falta porque el HUD y el tablero viven toda la ejecución. Un movimiento bloqueado no notifica, porque no cambió nada. `PantallaJuego` no es observador: después de cada movimiento consulta `getEstado()` para detectar la victoria. El sonido no es observador sino decorador (3.6): `JuegoConSonido.agregarObservador` delega en el juego envuelto.

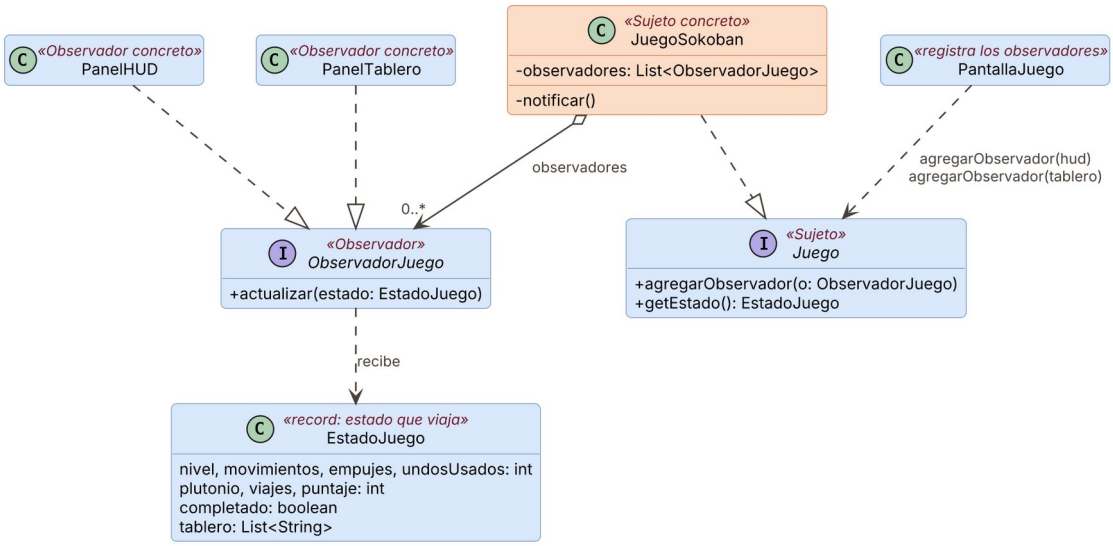


Figura 3. Observer: JuegoSokoban avisa a PanelHUD y PanelTablero a través de ObservadorJuego con un EstadoJuego.

3.3 Singleton: Imagenes y Sonidos

Problema. El tablero se redibuja en cada movimiento con decenas de imágenes y los 16 efectos de sonido pueden sonar muchas veces por minuto. Leer los archivos en cada uso sería lento y tener una copia por pantalla duplicaría memoria. Además, el estado de silencio debe ser uno solo para todo el juego.

Rol	Clases del proyecto
Singleton	`Imagenes`: atributo `static instancia`, constructor privado, `obtenerInstancia()` con creación diferida, clase `final`; caché `Map<String, Image>` con `computeIfAbsent`
Singleton	`Sonidos`: la misma estructura; el constructor carga un `Clip` por cada valor del enum `Sonido`
Clientes	`MenuPrincipal`, `PanelHUD`, `PanelTablero` (imágenes); `JuegoConSonido`, `SonidosDeInterfaz` (sonidos)

Por qué y no la alternativa. La alternativa más limpia es crear una única instancia en `Main` e inyectarla por constructor, lo que facilita las pruebas y respeta mejor DIP. Se eligió Singleton porque son recursos técnicos, globales por naturaleza (una caché y un dispositivo de audio), usados por componentes creados en distintos niveles de la jerarquía Swing: pasarlos por todos los constructores agregaría parámetros sin beneficio funcional.

Principios. **SRP**: cada clase tiene una sola responsabilidad técnica. GRASP: **Fabricación pura** y **Alta cohesión**. `Sonidos` además es **Facade** de `javax.sound` (`AudioSystem`, `Clip`, formatos y excepciones quedan detrás de `reproducir(Sonido.PASO)`) y contexto del State (3.7). Si la computadora no tiene salida de audio, captura el error, avisa por consola y el juego sigue en silencio. `Imagenes`, en cambio, falla con un mensaje claro si falta una imagen, porque sin ella el tablero no puede dibujarse.

Costos y límites. Es estado global: oculta dependencias y dificulta las pruebas. `obtenerInstancia()` **no es seguro con varios hilos** porque no está sincronizado. Se acepta porque todos los accesos ocurren en el hilo de eventos de Swing: `Main` usa `invokeLater`; teclas, botones y pintado corren en el EDT, y tanto el sonido de derrota como la repetición usan `javax.swing.Timer`, que también dispara en el EDT. Si hubiera otros hilos, convendría la inicialización anticipada o una clase contenedora estática.

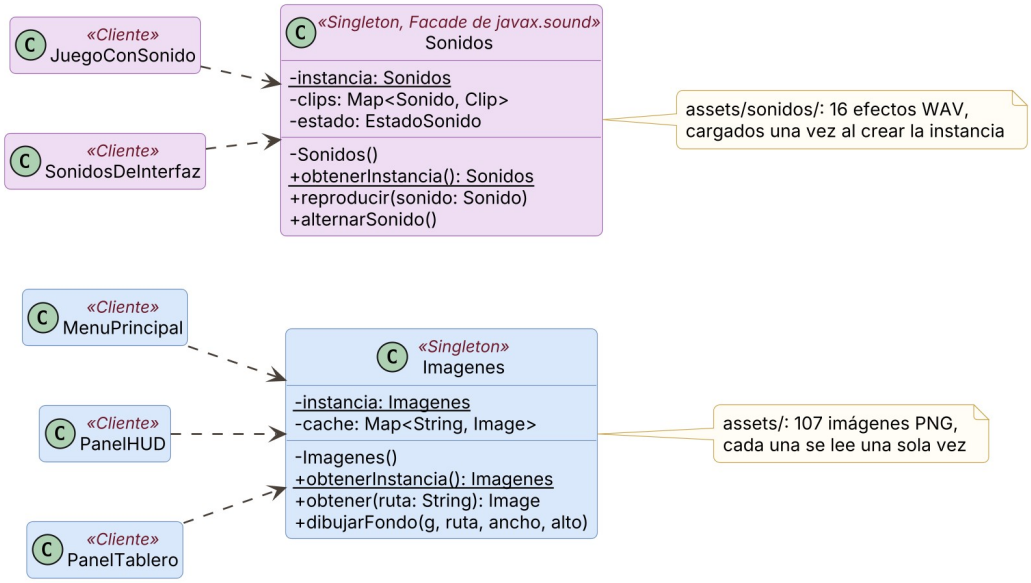


Figura 4. Singleton: Imagenes y Sonidos con sus clientes; cada archivo se carga una sola vez.

3.4 Strategy: los tres tipos de caja

Problema. Las cajas normales, frágiles y llave se mueven con las mismas reglas, pero después de un empuje reaccionan distinto: la frágil pierde resistencia y se rompe; la llave abre los muros si queda sobre un cerrojo. Sin el patrón, `Caja` o `MotorMovimiento` tendrían un condicional por tipo que crecería con cada caja nueva.

Rol	Clases del proyecto
Contexto	`Caja`: guarda `estrategia: ComportamientoCaja`; `despuesDeEmpujar(estado)` y `getSimbolo()` delegan en ella
Estrategia	`ComportamientoCaja`: `despuesDeEmpujar(caja, estado)` y `simbolo(caja)`
Estrategias concretas	`CajaNormalStrategy` (sin efecto, símbolo `$`); `CajaFragilStrategy` (resta resistencia y, al llegar a 0, `estado.quitarCaja(caja)`; símbolo `F`, o `f` cuando le queda 1); `CajaLlaveStrategy` (sobre un cerrojo, `estado.abrirMuros()`; símbolo `L`)
Quién elige	El constructor de `EstadoNivel`, según el carácter del archivo (`$`/`*`, `F` con resistencia 3, `L`)
Quién invoca	`MotorMovimiento`, después de ubicar la caja empujada

Por qué y no la alternativa. Con un enum de tipo y un `switch`, cada caja nueva obligaría a modificar `Caja` o el motor. Las subclases de `Caja` serían viables, pero mezclarían la parte común (posición y movimiento) con la reacción variable. Con estrategias sin estado (la resistencia vive en la `Caja`), `Caja.copiar()` comparte la misma estrategia al copiar el nivel para los recuerdos, sin copias polimórficas.

Principios. **OCP**: un tipo de caja nuevo es una clase nueva más un caso en `EstadoNivel`; `Caja` y `MotorMovimiento` no cambian. **SRP**, **LSP** y **DIP** (`Caja` depende de la interfaz). GRASP: **Polimorfismo**, **Experto en información** (cada estrategia conoce su efecto y su símbolo) y **Variaciones protegidas**.

Costos y límites. El constructor de `EstadoNivel` conoce las tres clases concretas; si los tipos crecieran, la creación podría pasar a un Factory Method. El símbolo de dibujo forma parte de la estrategia, un detalle de presentación dentro del dominio aceptado porque es el formato acordado del DTO. El atributo `resistencia` existe en todas las cajas aunque solo la frágil lo usa. La llave abre **todos** los muros X del nivel, no solo «los correspondientes»: con un cerrojo por nivel es equivalente.

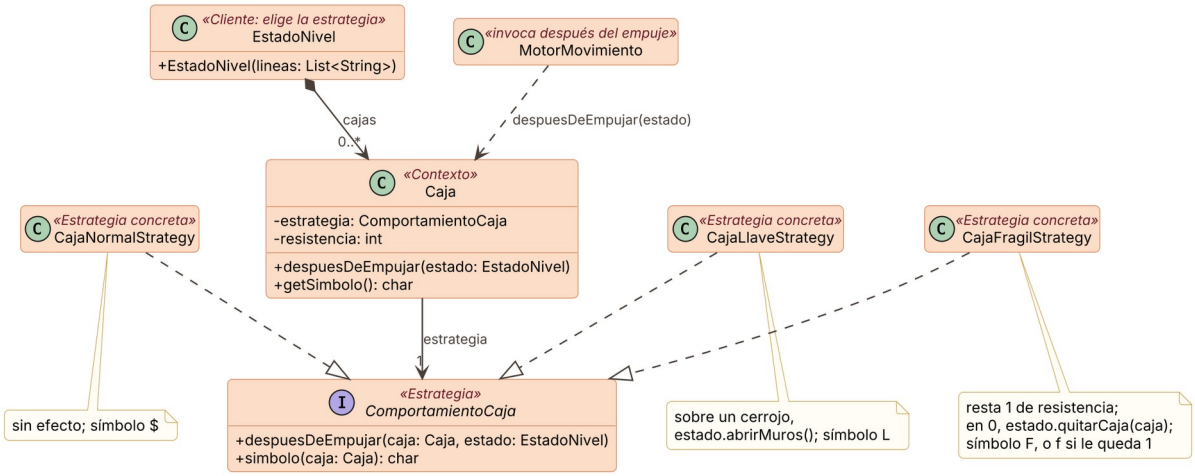


Figura 5. Strategy: Caja delega en ComportamientoCaja; las notas resumen lo que hace cada estrategia.

3.5 Mediator: el movimiento

Problema. Un paso involucra al jugador, a la caja que tiene adelante, a la celda detrás de la caja y al terreno (hielo, cerrojo, muros). Si el `Jugador` empujara a la `Caja` y cada uno consultara el tablero, las reglas quedarían repartidas y el jugador acoplado a las cajas.

Rol	Clases del proyecto
Mediador	`MediadorMovimiento`: `intentarMover(direccion): boolean`, `fueEmpuje(): boolean`
Mediador concreto	`MotorMovimiento`: colisiones, empuje de una sola caja, deslizamiento en hielo; conoce el `EstadoNivel`
Colega	`Jugador`: `mover(direccion)` delega en su mediador y solo conoce la interfaz
Objetos coordinados	`Caja` (el motor la mueve y le avisa con `despuesDeEmpujar`) y `EstadoNivel` (`cajaEn`, `estaLibre`, `esHielo`)
Conexión	`JuegoSokoban.conectarMotor()` crea un motor por nivel y lo conecta al jugador; se repite al restaurar un recuerdo (deshacer o DeLorean), porque el nivel restaurado trae un `Jugador` nuevo

Por qué y no la alternativa. Un `Jugador` con referencia al tablero que empuja cajas por su cuenta conocería cajas, paredes y terrenos, y cada regla nueva tocaría varias clases. Con el mediador, todas las reglas de movimiento están en `MotorMovimiento` y el `Jugador` quedó reducido a pedir el movimiento.

Principios. **SRP** y **DIP** (el `Jugador` depende de `MediadorMovimiento`). GRASP: **Indirección**, **Bajo acoplamiento** (`Jugador` y `Caja` no se conocen), **Fabricación pura** y **Alta cohesión**.

Costos y límites. En el Mediator del GoF varios colegas se comunican **a través** del mediador. Aquí **solo el `Jugador` lo usa**; la `Caja` es movida por el motor sin conocerlo. La implementación funciona como un mediador con un único colega que coordina al resto de los objetos, que nunca se hablan entre sí; para la forma canónica, la `Caja` también podría pedir su desplazamiento al mediador. Una regla de terreno nueva modifica `MotorMovimiento`. `fueEmpuje()` solo es válido inmediatamente después de `intentarMover` (acoplamiento temporal). `JuegoSokoban` guarda el motor como `MotorMovimiento` aunque solo usa la interfaz.

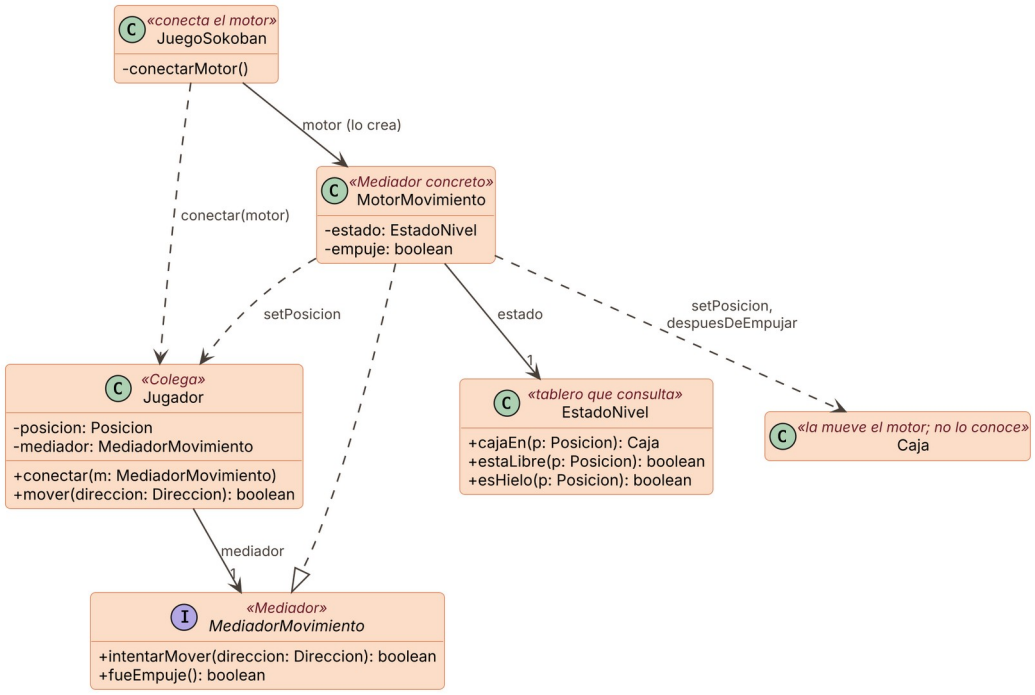


Figura 6. Mediator: el Jugador pide el movimiento a MediadorMovimiento; MotorMovimiento consulta el tablero y mueve la Caja.

3.6 Decorator: el sonido

Problema. La consigna pide que «las acciones del juego deben estar acompañadas de efectos de sonido apropiados». Poner llamadas de audio dentro de `JuegoSokoban` mezclaría reglas con sonido, y el sonido se desarrolló en paralelo, por otra integrante, sin tocar el dominio ni la vista.

Rol	Clases del proyecto
Componente	`Juego`
Componente concreto	`JuegoSokoban`
Decorador	`JuegoConSonido`: guarda `private final Juego juego` recibido por constructor; `iniciarNivel`, `mover`, `deshacer`, `reiniciar` y `viajarEnElTiempo` delegan y hacen sonar un efecto; `getCantidadNiveles`, `getEstado` y `agregarObservador` solo delegan
Cliente	La vista y los comandos, que reciben un `Juego`; el ensamblado está en `Main`

Para elegir el efecto, `mover` compara el `EstadoJuego` de antes y de después, en este orden de prioridad: sin cambios, choque; nivel completado, victoria; menos cajas frágiles, caja rota (y derrota 600 ms después si ya hay menos cajas que destinos); muros abiertos, cerrojo; más cajas sobre destino, «caja en destino»; más plutonio, plutonio; más empujes, empuje (o crujido si se movió una frágil); en otro caso, paso. `viajarEnElTiempo` suena como viaje en el tiempo, o como choque si no tuvo efecto.

Por qué y no la alternativa. Un observador de sonido solo se enteraría de los cambios notificados: no detectaría el choque contra una pared ni un deshacer sin efecto, y no distinguiría «iniciar nivel» de «reiniciar». Una subclase de `JuegoSokoban` quedaría atada a esa implementación; el decorador funciona con cualquier `Juego`. Además, como los comandos llaman al `Juego` decorado, la repetición de la partida suena igual que la partida original sin código adicional.

Principios. **OCP**: se agrega comportamiento sin modificar el dominio ni la vista, y se quita volviendo atrás una línea de `Main`. **SRP**, **LSP** y **DIP**. GRASP: **Variaciones protegidas** y **Bajo acoplamiento** (ni la vista ni el dominio importan `sokoban.sonido`).

Costos y límites. El decorador infiere los eventos comparando los símbolos del tablero, por lo que depende de la codificación del DTO. La «derrota» se detecta en el decorador y no en el dominio, que no tiene un estado de derrota; por eso solo suena un efecto. Cada operación nueva de `Juego` (como `viajarEnElTiempo`) obliga a agregarla también aquí. No hay decorador abstracto: con uno solo no hace falta.

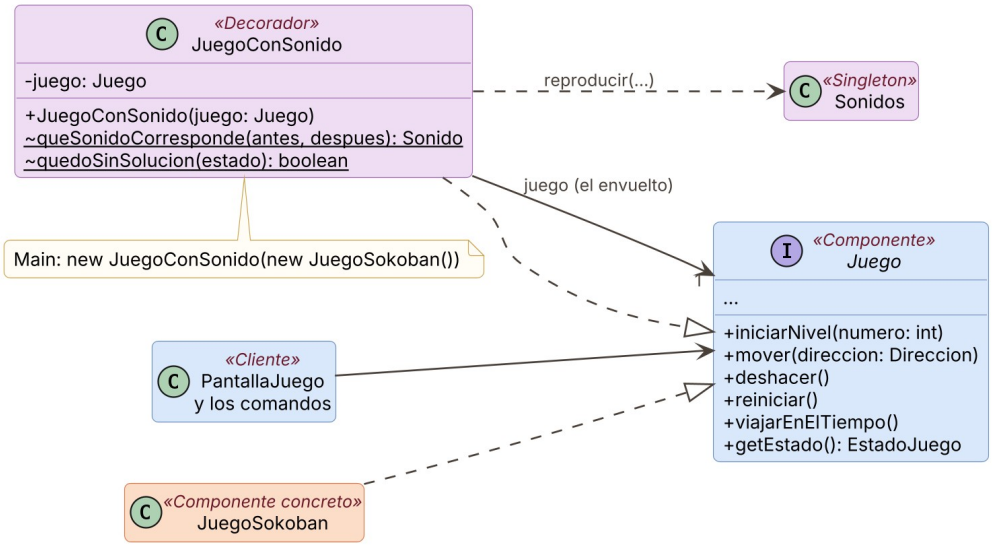


Figura 7. Decorator: JuegoConSonido implementa Juego y envuelve a otro Juego (hoy, JuegoSokoban).

3.7 State: sonido activado o silenciado

Problema. Con la tecla M el jugador silencia o activa el sonido. El mismo pedido, «reproducir este efecto», debe comportarse distinto según el estado.

Rol	Clases del proyecto
Contexto	`Sonidos`: guarda `estado: EstadoSonido` (inicial `SonidoActivado`); `reproducir` y `alternarSonido` delegan en él
Estado	`EstadoSonido` (visibilidad de paquete): `reproducir(Clip)` y `alternar(Sonidos)`
Estados concretos	`SonidoActivado` (detiene, rebobina e inicia el clip; al alternar corta todo y pasa a silenciado); `SonidoSilenciado` (no reproduce; al alternar pasa a activado)
Disparador	`SonidosDeInterfaz`: un `KeyEventDispatcher` global detecta la M sin consumir la tecla

Por qué y no la alternativa. Con un `boolean silenciado` y un condicional en cada método, cada estado nuevo (por ejemplo, volumen bajo) multiplicaría los condicionales. Con State cada estado sabe qué hacer y a cuál pasa. A diferencia de Strategy, las transiciones están dentro de los estados; en Strategy el cliente elige desde afuera (`EstadoNivel` asigna la estrategia de cada caja y nunca la cambia).

Principios. **OCP** (un estado nuevo no modifica `Sonidos.reproducir`), **SRP** y encapsulamiento: `establecerEstado` y `detenerTodo` son de paquete, así solo los estados cambian el estado. GRASP: **Polimorfismo** y **Experto en información**.

Costos y límites. Con dos estados, la estructura es mayor que un `boolean`; se justifica por la claridad de las transiciones. Cada cambio crea un objeto nuevo; como los estados no guardan datos, podrían reutilizarse dos instancias fijas. Un estado nuevo obliga a modificar el estado que transiciona hacia él.

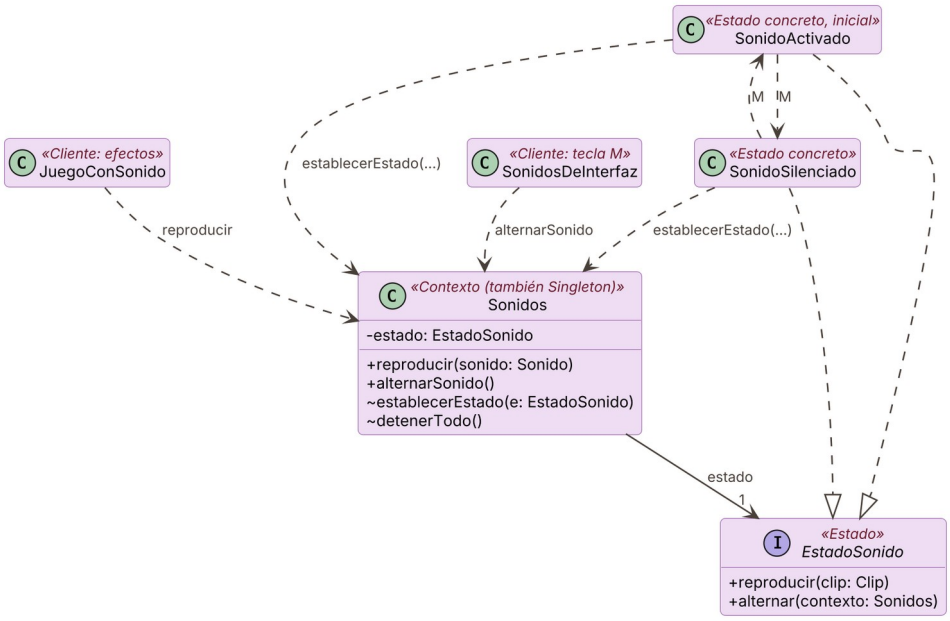


Figura 8. State: Sonidos delega en su estado actual; la tecla M dispara las transiciones.

3.8 Memento en el deshacer

La consigna establece: «Se permite deshacer los últimos 15 movimientos realizados por el jugador. Para ello, debe existir un botón undo que, cada vez que se presione, restaure el estado del nivel 5 movimientos hacia atrás, hasta un máximo de tres usos consecutivos.»

Funcionamiento. Antes de cada movimiento, `JuegoSokoban` crea un recuerdo con `crearRecuerdo()`. Si el movimiento se concreta, lo apila en `historial` (un `Deque<RecuerdoNivel>`) y, si supera `MAX_HISTORIAL = 15`, descarta el más viejo. `deshacer()` no hace nada si el nivel está completo, si el historial está vacío o si ya se usó `MAX_UNDOS_CONSECUTIVOS = 3` veces seguidas; si no, saca hasta `PASOS_POR_UNDO = 5` recuerdos, restaura el último que sacó (tablero, movimientos, empujes y plutonio), descarta los puntos del DeLorean que quedaron «en el futuro», suma un deshacer usado y notifica. El contador de usos consecutivos vuelve a cero con cada movimiento efectivo.

Rol	Clases del proyecto
Originador	`JuegoSokoban`: `crearRecuerdo()` y `restaurar(recuerdo)`; el estado guardado es el `EstadoNivel` más los contadores, que viven en `JuegoSokoban`
Memento	`RecuerdoNivel` (`public final`): copia profunda del nivel (`EstadoNivel.copiar()`), movimientos, empujes y plutonio; constructor y lectores con visibilidad de paquete; `nivel()` devuelve una copia nueva para poder restaurar el mismo recuerdo más de una vez
Cuidadores	El `historial` de `JuegoSokoban` (deshacer) y `MaquinaDelTiempo` (DeLorean, sección 6.1), que está en otro paquete y no puede leer el recuerdo

Por qué y no la alternativa. La alternativa habitual es Command con una operación inversa por movimiento. Aquí deshacer un empuje implicaría revertir deslizamientos sobre el hielo, resistencias de cajas frágiles, cajas rotas que desaparecieron, muros abiertos por la llave y plutonio recogido. Un recuerdo completo restaura todo sin lógica inversa, con menos riesgo de errores, y el mismo `RecuerdoNivel` sirve para el deshacer y para el DeLorean. El costo de memoria (hasta 15 copias de un nivel chico, más los puntos del DeLorean) es despreciable.

Principios. Encapsulamiento: nadie fuera de `sokoban.modelo` puede leer ni modificar un recuerdo. **SRP**: `RecuerdoNivel` solo guarda un momento. GRASP **Experto en información** (`EstadoNivel` sabe copiarse) y **Creador** (`JuegoSokoban` crea los recuerdos porque tiene los datos para inicializarlos).

Costos y límites. El encapsulamiento es por paquete: cualquier clase de `sokoban.modelo` podría leer un recuerdo. Para el deshacer, `JuegoSokoban` es a la vez originador y cuidador (la pila es privada). Restaurar exige reconectar el motor porque el `Jugador` copiado no tiene mediador.

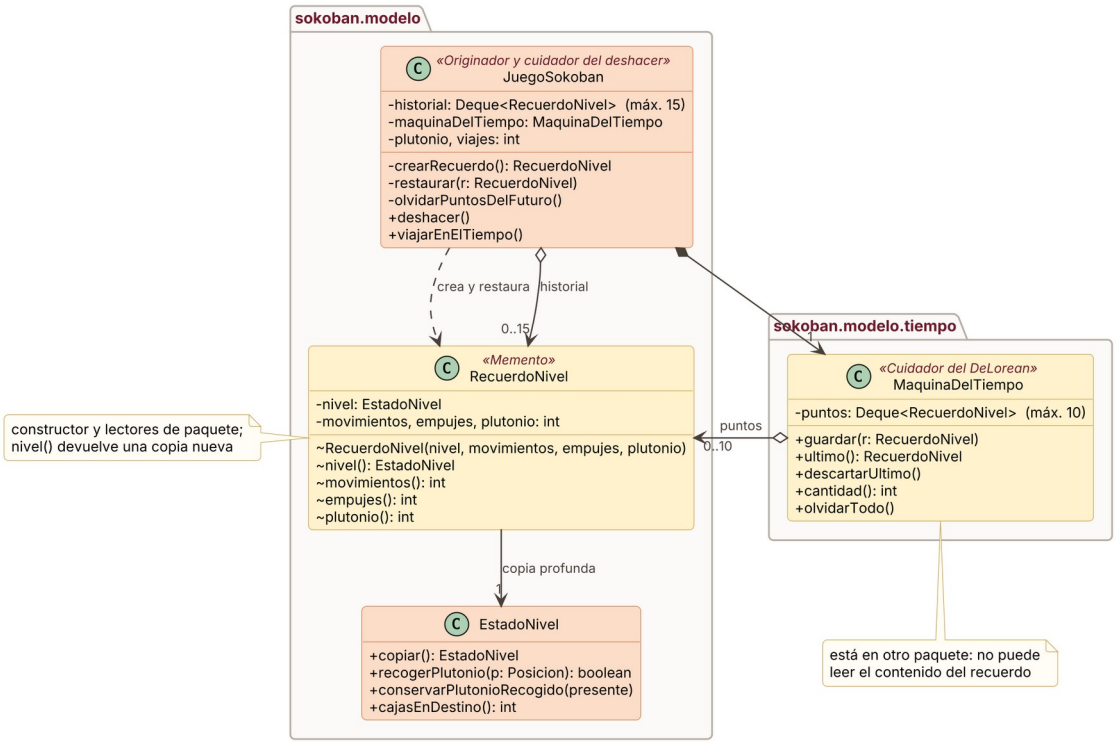


Figura 9. Memento: JuegoSokoban crea y restaura RecuerdoNivel; los guardan el historial del deshacer y MaquinaDelTiempo.

3.9 Resumen de principios SOLID y GRASP

La tabla reúne la evidencia de cada principio en el código y el punto débil que el equipo reconoce.

Principio	Evidencia en el código	Punto débil reconocido
SRP	Leer el archivo (`CargadorNivel`), guardar el tablero (`EstadoNivel`), mover (`MotorMovimiento`), reaccionar al empuje (estrategias), reproducir (`Sonidos`), guardar un momento (`RecuerdoNivel`), grabar acciones (`GrabadoraDePartida`).	`PantallaJuego` arma la partida, maneja el teclado, crea los comandos, controla la repetición y muestra carteles; `JuegoSokoban` suma historial, contadores y puntaje.
OCP	Caja nueva = estrategia nueva; acción nueva = comando nuevo; sonido = decorador; observador, época o símbolo nuevos sin tocar la lógica existente.	Una caja nueva suma un caso en `EstadoNivel`; el DeLorean agregó un método a `Juego`.
LSP	`JuegoSokoban` y `JuegoConSonido` son intercambiables detrás de `Juego`; las estrategias, los estados y los comandos, detrás de sus interfaces.	Sin observaciones.
ISP	`ObservadorJuego` y `Comando` tienen un método; `MediadorMovimiento`, `ComportamientoCaja` y `EstadoSonido`, dos. `PanelHUD` recibe solo `Runnable`.	`Juego` tiene ocho métodos y mezcla acciones con el registro de observadores.
DIP	La vista y los comandos dependen de `Juego`; el `Jugador`, de `MediadorMovimiento`; la `Caja`, de `ComportamientoCaja`; el invocador, de `Comando`. Solo `Main` conoce las clases concretas.	`JuegoSokoban` guarda el motor con su tipo concreto; los Singleton se usan por acceso estático.
Experto	`EstadoNivel` sabe si una celda está libre, si se ganó y cuántas cajas hay en destino; `Direccion` sabe cuánto se desplaza.	
Creador	`EstadoNivel` crea el `Jugador` y las `Caja`; `CargadorNivel` crea el `EstadoNivel`; `JuegoSokoban` crea los `RecuerdoNivel`; `PantallaJuego` crea los comandos.	

Principio	Evidencia en el código	Punto débil reconocido
Controlador	`JuegoSokoban` recibe los eventos del sistema y coordina.	La entrada de teclado vive en la vista (habitual en Swing).
Bajo acoplamiento y Alta cohesión	Vista, dominio y sonido no se conocen; `PanelHUD` ya no conoce `Juego`; clases cortas con una tarea (la más larga, `PantallaJuego`, tiene 210 líneas con comentarios).	`JuegoConSonido` depende de los símbolos del DTO.
Polimorfismo	Cajas, estados del sonido, observadores y comandos se usan sin preguntar su clase.	
Fabricación pura	`Imagenes`, `Sonidos`, `CargadorNivel`, `MotorMovimiento`, `MaquinaDelTiempo`, `GrabadoraDePartida`, `Repeticion`.	
Indirección y Variaciones protegidas	`Juego` entre la vista y el dominio; `MediadorMovimiento` entre el jugador y las reglas; `Comando` entre la entrada y la ejecución.	

4. Reglas del juego implementadas

La tabla relaciona cada requisito funcional de la consigna con su implementación. Todos se probaron en pantalla con los tres niveles incluidos.

Requisito de la consigna	Implementación	Clases
Tablero: paredes, vacíos, cajas, destinos, hielo, cerrojos, muros, jugador	Un símbolo por celda (4.1); cada elemento tiene su imagen.	`EstadoNivel`, `PanelTablero`
Niveles desde archivos .txt	Lee `niveles/nivelN.txt` desde el `.jar` o, si no está, desde la carpeta.	`CargadorNivel`
Movimiento con flechas; solo empujar; una caja a la vez; no atravesar cajas, paredes ni muros cerrados	La celda siguiente debe estar libre; si hay una caja, la celda detrás de ella también.	`PantallaJuego`, `MotorMovimiento`, `EstadoNivel.estaLibre`
Caja normal, frágil y llave	Estrategias (3.4); la frágil resiste 3 empujes.	`Caja`, `estrategias.*`
Terreno resbaladizo	La caja se desliza en la dirección del empuje (4.3).	`MotorMovimiento`
Deshacer 15 / 5 / 3	Recuerdos del nivel (3.8).	`JuegoSokoban`, `RecuerdoNivel`
Victoria y avance de nivel	Todas las cajas en destinos; cartel con «Siguiente nivel».	`EstadoNivel.estaCompleto`, `PantallaJuego`
Resumen y puntaje	Movimientos, empujes, deshacer, viajes y puntaje al completar (sección 5).	`JuegoSokoban.getEstado`, `PantallaJuego`
HUD con movimientos, empujes y nivel; botones deshacer y reiniciar	Indicadores y botones con los íconos del diseño.	`PanelHUD`
Imágenes y sonidos de dominio público	107 PNG propios del equipo; 16 WAV generados por código.	`Imagenes`, `Sonidos`
Dos funcionalidades adicionales con patrones	DeLorean (Memento) y repetición de la partida (Command).	Sección 6

4.1 Formato de los niveles y validación

Cada nivel es un archivo de texto con un carácter por celda y todas las filas del mismo ancho:

Carácter	Elemento	Carácter	Elemento
#	Pared	F	Caja frágil (resistencia 3)
(espacio)	Piso	L	Caja llave
.	Destino	~	Terreno resbaladizo
@ / +	Jugador / jugador sobre destino	C	Cerrojo
$ / *	Caja normal / caja normal sobre destino	X / O	Muro cerrado / muro abierto
		P	Plutonio: una carga para el DeLorean

El constructor de `EstadoNivel` valida el archivo y lanza `IllegalArgumentException` con un mensaje claro si el nivel está vacío, si hay filas de distinto ancho, si aparece un símbolo desconocido, si falta el jugador o hay más de uno, o si la cantidad de destinos no coincide con la de cajas (o no hay destinos). Un error de lectura se informa como `IllegalStateException`. Los símbolos `f`, `G`, `g`, `K` y `c` no se escriben en los archivos: los genera el dominio para la vista. La cantidad de niveles está fijada en la constante `NIVELES = 3`, y cada nivel incluye un plutonio.

4.2 Tipos de caja

- **Normal:** se empuja en la dirección del jugador y no tiene efecto adicional.
- **Frágil:** empieza con resistencia 3 y cada empuje la reduce en 1. Cuando le queda 1 se dibuja dañada; al llegar a 0 se rompe y desaparece del tablero. Como `estaCompleto()` exige tantas cajas como destinos, romper una caja necesaria deja el nivel sin solución: suena el efecto de derrota y el jugador debe reiniciar o viajar con el DeLorean.
- **Llave:** al quedar sobre un cerrojo `C` ejecuta `abrirMuros()`, que convierte todos los muros `X` en `O` y el cerrojo en abierto. Los muros quedan abiertos aunque la caja se mueva después.

4.3 Terreno resbaladizo

Cuando una caja empujada entra al hielo, sigue avanzando en la misma dirección mientras la celda siguiente esté libre. Se detiene en la primera celda que no es hielo o, si encuentra un obstáculo, en la última celda de hielo. El deslizamiento cuenta como un solo empuje y el efecto de la estrategia se aplica una vez, en la posición final. El jugador camina sobre el hielo sin deslizarse; la consigna exige el deslizamiento solo para las cajas.

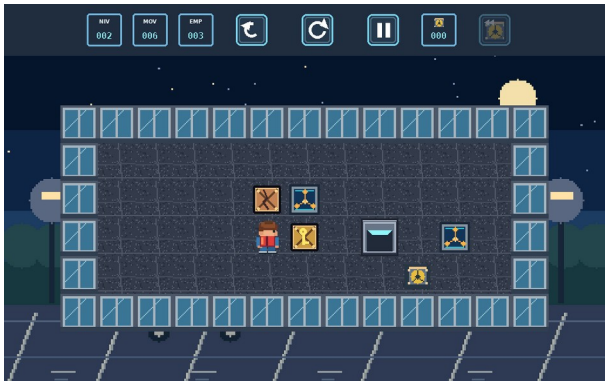
4.4 Deshacer, victoria e interfaz

Deshacer. Cumple la regla 15 / 5 / 3 como se describe en 3.8. Si quedan menos de cinco recuerdos, vuelve al más antiguo. Restaura también el plutonio: si se deshace el paso en que se recogió, vuelve a aparecer en el tablero. Cada uso suma una penalización al puntaje y no se puede deshacer una vez completado el nivel.

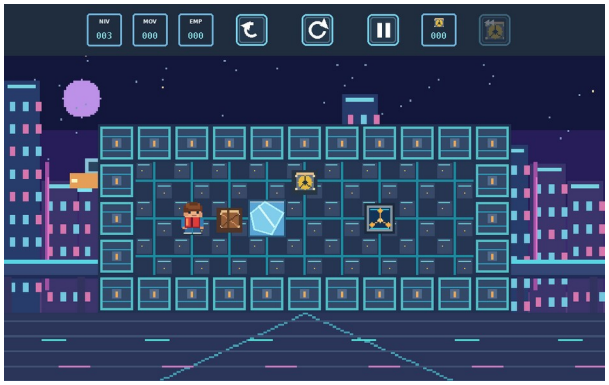
Victoria. El nivel se completa cuando todas las cajas están sobre destinos. A partir de ese momento se ignoran los movimientos, el deshacer y el DeLorean, y `PantallaJuego` muestra un cartel modal «¡NIVEL COMPLETADO!» con movimientos, empujes, deshacer, viajes en el tiempo y puntaje, y las opciones «Siguiente nivel», «Ver repetición», «Repetir» y «Menú». El avance ocurre al elegir «Siguiente nivel». En el último nivel el cartel se titula «RESULTADOS» y no ofrece el siguiente.

HUD e interfaz. `PanelHUD` muestra el nivel, los movimientos, los empujes y las cargas de plutonio con tres dígitos, y los botones Deshacer, Reiniciar, Pausa y DeLorean; este último se ve apagado si no hay cargas o el nivel está completo. Los botones no toman el foco del teclado, así las flechas siempre mueven al jugador, y

tienen texto de ayuda y nombre accesible. Esc o el botón de pausa abren un cartel con «Continuar», «Reiniciar» y «Menú». Mientras un cartel está abierto, el juego no recibe teclas.



a) Nivel 2 tras seis movimientos: la frágil recibió un empuje, la llave está sobre el cerrojo y el muro quedó abierto. Sin plutonio, el botón del DeLorean está apagado.



b) Nivel 3 al comenzar: la caja está junto al terreno resbaladizo; arriba, el plutonio.

Figura 10. Capturas del juego. Arriba, el HUD: nivel, movimientos, empujes, Deshacer, Reiniciar, Pausa, cargas de plutonio y DeLorean.

4.5 Sonidos

Los 16 efectos están en `assets/sonidos/` y se generan por código con `docs/generar_sonidos.py` (ondas y ruido sintetizados), por lo que son propios y de dominio público. La tecla M silencia o activa el sonido; si la computadora no tiene salida de audio, el juego sigue sin sonido.

Situación	Efecto	Situación	Efecto
Se abre el juego	`ARRANQUE`	Se abren los muros	`CERROJO_ABIERTO`
Empieza un nivel	`INICIO_NIVEL`	Deshacer	`DESHACER`
Paso del jugador	`PASO`	Reiniciar el nivel	`REINICIAR`
Empuje de una caja	`EMPUJE`	Nivel ganado	`VICTORIA`
Caja sobre su destino	`CAJA_EN_DESTINO`	Nivel sin solución	`DERROTA`
Caja frágil empujada	`CAJA_DANADA`	Clic en cualquier botón	`CLIC`
Caja frágil rota	`CAJA_ROTA`	Juntar plutonio	`PLUTONIO`
Movimiento imposible	`CHOQUE`	Viaje en el tiempo	`VIAJE_EN_EL_TIEMPO`

5. Criterio de puntaje

La consigna pide mostrar al final de cada nivel un resumen con movimientos, empujes, uso del deshacer y puntaje, y deja el criterio a definición del equipo. El puntaje se calcula en `JuegoSokoban.getEstado()`, viaja en `EstadoJuego` y se muestra en el cartel de resultado:

```
puntaje = max(0, 1000 − 10·movimientos − 5·empujes − 50·deshacer − 100·viajes)
```

Justificación de cada término:

- **Base de 1000 puntos:** una escala fácil de leer en la que cada penalización es visible.
- **10 puntos por movimiento:** la consigna describe el objetivo como completar cada nivel «de la manera más eficiente posible»; menos pasos, más puntos.
- **5 puntos adicionales por empuje:** un empuje también cuenta como movimiento, así que cuesta 15 en total. Los empujes son las acciones que modifican el tablero y pueden trabar una caja; penalizarlos

premia las soluciones más limpias. Movimientos y empujes son las dos métricas clásicas con que se comparan soluciones de Sokoban.

- **50 puntos por deshacer:** al deshacer, los movimientos y empujes vuelven a los valores del recuerdo y dejan de contar. Sin penalización, deshacer serviría para «limpiar» el contador gratis.
- **100 puntos por viaje en el tiempo:** el DeLorean puede volver más atrás que el deshacer y rescatar un nivel sin solución, y también devuelve los contadores a los del punto en el tiempo; por ser más poderoso, cuesta el doble que un deshacer.
- **Piso de 0:** evita puntajes negativos en partidas muy largas.

Reiniciar el nivel pone todos los contadores en cero, incluidos los de deshacer, viajes y plutonio. El criterio no considera el tiempo de juego, que no se registra.

Partida	Mov.	Emp.	Desh.	Viajes	Puntaje
Nivel 1 resuelto en el mínimo de pasos	3	2	0	0	1000 − 30 − 10 = **960**
Nivel 2, situación de la Figura 10 a)	6	3	0	0	1000 − 60 − 15 = **925**
Nivel 2, después del viaje de la Figura 11 b)	21	2	0	1	1000 − 210 − 10 − 100 = **680**
Ejemplo ilustrativo con un deshacer	20	6	1	0	1000 − 200 − 30 − 50 = **720**

Los tres primeros valores se obtuvieron ejecutando el juego; el cuarto es un cálculo de la fórmula.

6. Funcionalidades adicionales

La consigna exige «incorporar dos funcionalidades adicionales al juego que impliquen la aplicación de patrones de diseño». El equipo implementó dos que se apoyan en el diseño existente: **DeLorean**, para volver atrás en el tiempo hasta un punto guardado (Memento), y **Repetición de la partida**, para ver el nivel jugado al terminarlo (Command).

6.1 DeLorean: volver atrás en el tiempo (Memento)

Descripción. Cada nivel tiene un plutonio (`P`); al pisarlo, el jugador suma una carga, que el HUD muestra. El juego guarda solo **puntos en el tiempo**: al empezar el nivel y cada vez que aumenta la cantidad de cajas sobre destinos (hasta 10; se descarta el más viejo). Con la tecla T o el botón DeLorean, el jugador gasta una carga y vuelve al último punto. Si ese punto es el presente (no hubo movimientos desde que se guardó), vuelve al anterior; si no hay un punto anterior, no pasa nada y no se gasta la carga. El plutonio ya recogido no reaparece, las cargas gastadas no se recuperan, el historial del deshacer se vacía y cada viaje resta 100 puntos.

Problema que resuelve. Un nivel que quedó sin solución, por ejemplo al romper una caja frágil necesaria, antes solo podía recuperarse reiniciando, y el deshacer alcanza como máximo a los últimos 15 movimientos con tres usos seguidos. El DeLorean recupera un momento clave sin perder todo el progreso, como un recurso que se gana jugando.

Rol	Clases del proyecto
Originador	`JuegoSokoban`: `crearRecuerdo()`, `restaurar(recuerdo)`, `viajarEnElTiempo()` y `olvidarPuntosDelFuturo()`; decide cuándo guardar un punto porque conoce los contadores y sabe cuándo llegó una caja a destino
Memento	`RecuerdoNivel`, el mismo del deshacer (3.8)
Cuidador	`MaquinaDelTiempo` (`sokoban.modelo.tiempo`): `guardar`, `ultimo`, `descartarUltimo`, `cantidad`, `olvidarTodo`; `MAXIMO_PUNTOS = 10`. Al estar en otro paquete, no puede leer el contenido de los recuerdos
Apoyo en el dominio	`EstadoNivel`: `recogerPlutonio(p)`, `conservarPlutonioRecogido(presente)`, `cajasEnDestino()`

Rol	Clases del proyecto
Contrato, vista y sonido	`Juego.viajarEnElTiempo()`; `EstadoJuego.plutonio()` y `viajes()`; indicador y botón en `PanelHUD`; tecla T y `ComandoViajarEnElTiempo` en `PantallaJuego`; efectos `PLUTONIO` y `VIAJE_EN_EL_TIEMPO` en `JuegoConSonido`

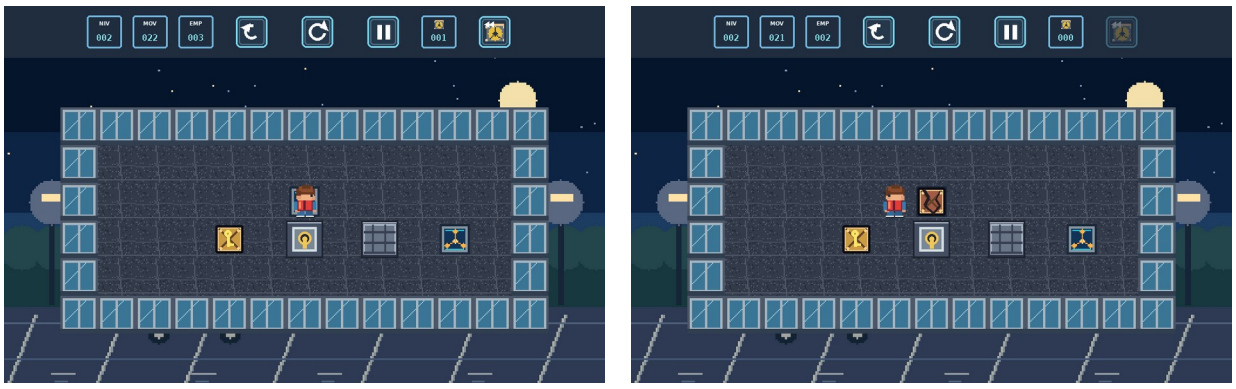
Por qué el patrón y no una alternativa. Un punto en el tiempo tiene que capturar todo el nivel (posiciones, resistencias, cajas rotas, muros abiertos) y restaurarlo mucho después, sin que quien lo guarda dependa de la estructura interna. Memento resuelve exactamente eso. Reconstruir el pasado con operaciones inversas sería complejo (3.8), y volver a ejecutar los comandos grabados desde el principio acoplaría el DeLorean a la repetición y sería más lento. Reutilizar `RecuerdoNivel` deja un único mecanismo para el deshacer y el DeLorean.

Principios. **SRP**: `MaquinaDelTiempo` solo administra puntos; `RecuerdoNivel` solo guarda un momento. Encapsulamiento del estado del nivel. GRASP: **Experto en información** y **Creador** (`JuegoSokoban`), **Fabricación pura** (`MaquinaDelTiempo`) y **Bajo acoplamiento** (el cuidador no conoce `EstadoNivel`). La vista y el sonido se extendieron sin cambiar su estructura, gracias a Facade, Observer y Decorator.

Diferencias con el deshacer obligatorio:

Aspecto	Deshacer (consigna)	DeLorean
Propósito	Corregir errores recientes	Salir de situaciones sin solución sin reiniciar
A dónde vuelve	5 movimientos atrás, dentro de los últimos 15	Al último punto en el tiempo (inicio del nivel o última caja que llegó a destino)
Disponibilidad	Siempre, hasta 3 usos seguidos	Una carga de plutonio por viaje
Cuidador	Pila `historial` de `JuegoSokoban` (15)	`MaquinaDelTiempo` (10)
Plutonio	Se restaura: el plutonio recogido vuelve al tablero	No reaparece; se conservan las cargas menos la usada
Efecto sobre el otro	Descarta los puntos del futuro	Vacía el historial del deshacer
Puntaje	−50 por uso	−100 por viaje

Costos y límites. Agregar `viajarEnElTiempo` a `Juego` obligó a modificar sus dos implementaciones. Para saber si el último punto es el presente se comparan los movimientos, lo cual es válido porque el juego es determinista. Solo se guardan puntos al inicio y cuando una caja llega a destino; si ninguna llegó, el único destino posible es el comienzo. El encapsulamiento del recuerdo es por paquete.



a) Nivel 2: el tercer empuje rompió la caja frágil y el nivel quedó sin solución. El jugador tiene una carga de plutonio.

b) Tras viajar con el DeLorean: la frágil vuelve a su destino, dañada, como en el último punto en el tiempo; las cargas quedan en 0 y el botón se apaga.

Figura 11. El DeLorean rescata un nivel sin solución sin reiniciarlo.

6.2 Repetición de la partida (Command)

Descripción. Cada acción del jugador durante el nivel (mover, deshacer y viajar en el tiempo) se convierte en un objeto comando que la `GrabadoraDePartida` ejecuta y anota. Reiniciar no se graba: borra la grabación, de modo que la repetición muestra la partida desde el último comienzo. Al completar el nivel, «Ver repetición» deja el nivel como al principio y `Repeticion` vuelve a ejecutar los comandos uno por uno cada 160 ms. Un aviso indica «REPETICIÓN • Esc: saltar al final»; mientras dura, se ignoran las acciones del jugador y Esc ejecuta de golpe lo que falta. Al terminar, vuelve a aparecer el cartel de resultado. Como el juego no tiene azar, los mismos comandos llevan exactamente a la misma partida; se verificó con una partida de 34 comandos que terminó en el mismo estado.

Problema que resuelve. Para repetir una partida hay que conservar qué pidió el jugador y en qué orden, independientemente de la tecla o el botón que originó el pedido, y poder ejecutarlo de nuevo con la lógica real del juego.

Rol	Clases del proyecto
Comando	`Comando`: `ejecutar()`
Comandos concretos	`ComandoMover(juego, direccion)`, `ComandoDeshacer(juego)`, `ComandoViajarEnElTiempo(juego)`
Invocador	`GrabadoraDePartida`: `ejecutar(comando)` ejecuta y anota; `borrar()`; `getGrabados()` devuelve una copia
Reproductor	`Repeticion`: vuelve a invocar los comandos con un `javax.swing.Timer`; `empezar()`, `saltarAlFinal()`, `estaReproduciendo()`
Receptor	`Juego`; en ejecución, el `JuegoConSonido` que envuelve a `JuegoSokoban`
Cliente	`PantallaJuego`: crea los comandos a partir de las flechas, la tecla T y los *callbacks* de `PanelHUD`

Por qué el patrón y no una alternativa. Guardar una imagen o un `EstadoJuego` por paso mostraría la partida como fotos, sin pasar por las reglas ni por el sonido. Guardar las teclas presionadas ataría la repetición a Swing y a la vista. Con Command, cada pedido es un objeto independiente de cómo se originó, que puede anotarse y volver a ejecutarse sobre el juego real: la repetición recorre el mismo dominio, los observadores animan el tablero y el decorador hace sonar los mismos efectos.

Principios. **OCP**: una acción nueva es un comando nuevo; `GrabadoraDePartida` y `Repeticion` no cambian. **SRP**: la grabadora anota, la repetición programa la reproducción y cada comando encapsula un pedido. **DIP**: los comandos dependen de `Juego` y el invocador, de `Comando`. GRASP: **Polimorfismo**, **Indirección**, **Creador** (`PantallaJuego` tiene los datos para crear cada comando) y **Bajo acoplamiento** (`PanelHUD` dejó de conocer a `Juego`: recibe `Runnable`).

Relación con el resto del diseño. Command es la alternativa clásica para implementar deshacer; aquí el deshacer sigue con Memento y Command se usa para grabar y repetir. El deshacer y el viaje en el tiempo también se graban como comandos, así que la repetición los reproduce.

Costos y límites. La repetición se ejecuta sobre el juego real y no sobre una copia: depende de que el juego sea determinista y de que el nivel empiece igual. `PantallaJuego` sumó responsabilidades (cliente de Command, control de la repetición y aviso), lo que la aleja de SRP. Los comandos no implementan una operación inversa porque no la necesitan.

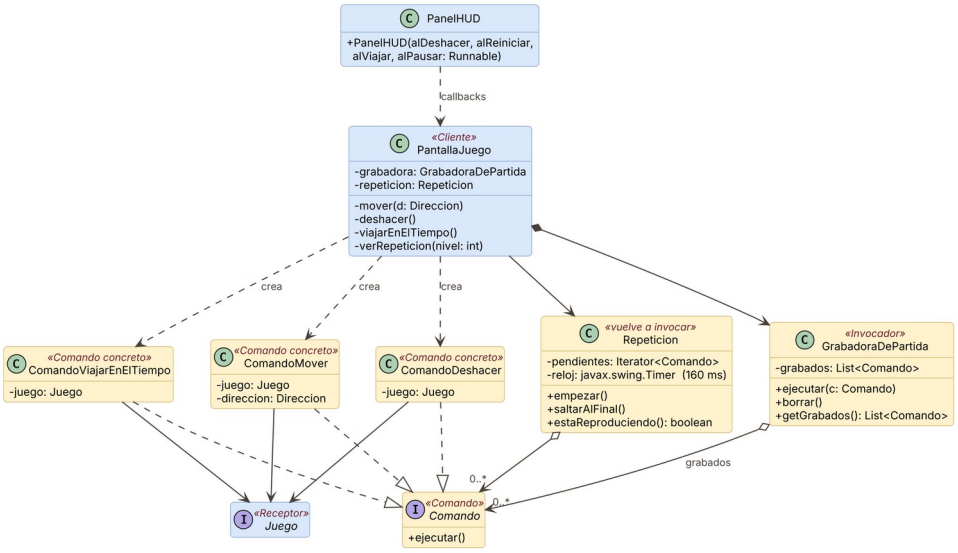


Figura 12. Command: PantallaJuego crea los comandos; GrabadoraDePartida los ejecuta y anota; Repeticion los vuelve a ejecutar sobre Juego.

7. Diagramas

Los diagramas de este informe se generaron con PlantUML a partir del código y usan colores para indicar el autor de cada clase: azul, Luca; naranja, Pedro y Lucía; lila, Mora; amarillo, Martín (integración y funcionalidades adicionales). El diagrama de clases en StarUML que pide la consigna se entrega en el repositorio como `docs/Sokoban.mdj`.

7.1 Diagramas de patrones

Cada patrón tiene su diagrama junto a su justificación: la Figura 1 muestra los paquetes; las Figuras 2 a 9 corresponden, en orden, a Facade, Observer, Singleton, Strategy, Mediator, Decorator, State y Memento (sección 3), y la Figura 12, a Command (sección 6.2). En cada una, el estereotipo sobre el nombre de la clase indica el rol que cumple en el patrón.

7.2 Diagrama de secuencia: mover y empujar una caja

La Figura 13, en orientación horizontal en la página siguiente, sigue una pulsación de la flecha derecha que empuja una caja:

1. `PantallaJuego` orienta el sprite, crea un `ComandoMover` y se lo da a la `GrabadoraDePartida` (Command), que lo ejecuta y lo anota.
2. El comando llama a `mover` sobre el `Juego`: atiende primero `JuegoConSonido` (Decorator), que guarda el estado previo y delega en `JuegoSokoban` (Facade).
3. `JuegoSokoban` crea un `RecuerdoNivel` (Memento) y le pide al `Jugador` que se mueva; el `Jugador` delega en `MotorMovimiento` (Mediator).
4. El motor verifica la celda, ubica la caja (deslizándola si cae en hielo), le avisa con `despuesDeEmpujar` (Strategy) y mueve al jugador.
5. `JuegoSokoban` apila el recuerdo, actualiza contadores, recoge plutonio, guarda un punto en el tiempo si una caja llegó a destino y notifica a `PanelHUD` y `PanelTablero` (Observer).
6. `JuegoConSonido` compara el estado anterior con el nuevo y reproduce el efecto con `Sonidos` (Singleton). Por último, `PantallaJuego` consulta `getEstado()` y, si el nivel quedó completo, muestra el resultado.

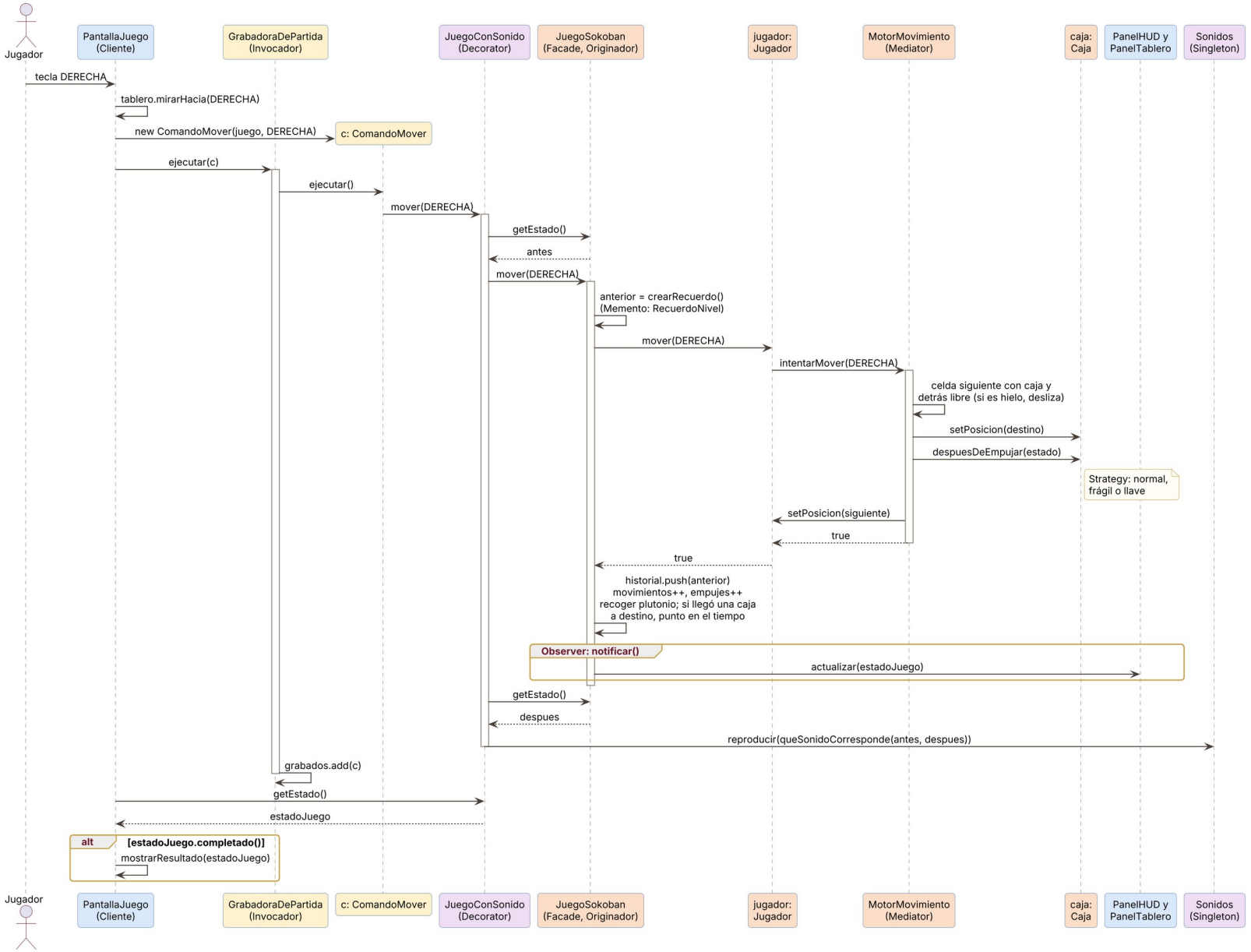


Figura 13. Diagrama de secuencia de un movimiento con empuje. El color de cada participante indica su autor.

7.3 Diagrama de clases completo

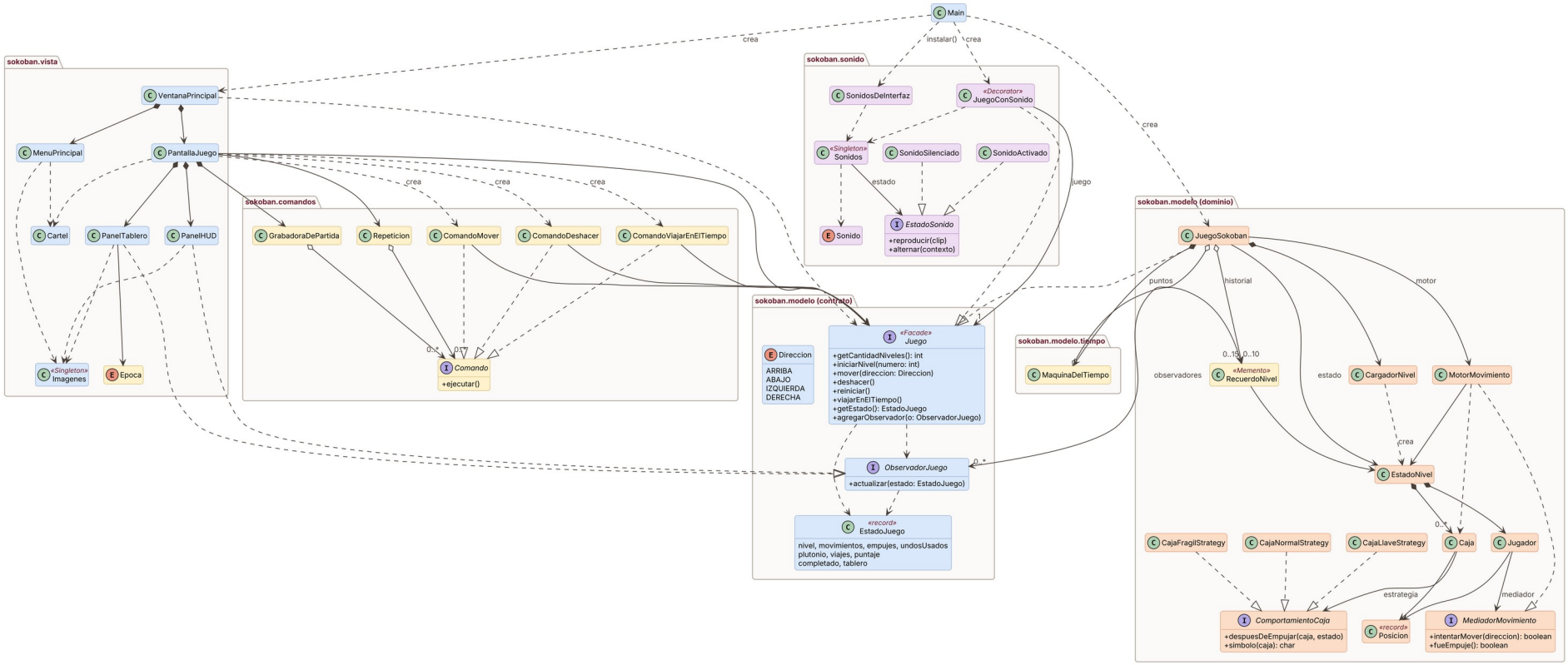


Figura 14. Diagrama de clases completo: todas las clases, agrupadas por paquete, con sus relaciones y las operaciones de las interfaces. Los atributos y métodos de cada clase están en las figuras de los patrones y en docs/Sokoban.mdj.

8. Limitaciones conocidas y trabajo futuro

Las siguientes observaciones surgen de la revisión del código integrado. Ninguna impide jugar ni incumple la consigna; se documentan para que el diseño pueda defenderse con sus límites a la vista.

Limitación	Consecuencia	Mejora propuesta
Solo el `Jugador` usa el mediador; la `Caja` es movida por el motor sin conocerlo.	Forma menos canónica del Mediator.	Que la `Caja` también pida su desplazamiento al mediador, o documentarlo como mediador con un solo colega.
Al romperse una caja frágil necesaria solo suena «derrota»; no hay aviso en pantalla.	El jugador debe deducir que tiene que reiniciar o viajar con el DeLorean.	Estado de derrota en el dominio y en `EstadoJuego`, con un cartel en `PantallaJuego`.
La caja llave sobre el cerrojo oculta el cerrojo; el jugador o una caja sobre hielo ocultan el hielo.	Se pierde información visual.	Símbolos adicionales o un DTO por capas.
Los tres niveles son chicos; la cantidad está fijada en `NIVELES = 3`.	Poca demostración de las reglas; sumar un nivel requiere modificar el código.	Diseñar niveles que muestren cada regla y detectar los archivos disponibles.
La llave abre todos los muros `X` del nivel.	Correcto con un único cerrojo por nivel.	Asociar cada cerrojo con sus muros.
Un archivo de nivel inválido lanza una excepción que la vista no informa.	La acción se interrumpe sin aviso al usuario.	Capturar el error en la vista y mostrarlo en un `Cartel`.
`JuegoConSonido` infiere los eventos por los símbolos del tablero.	Cambiar una letra en el dominio exige actualizar el sonido.	Que el DTO informe los eventos ocurridos en el movimiento.
`Juego` tiene ocho métodos y mezcla acciones con el registro de observadores; notificar es una convención.	Cada operación nueva modifica todas las implementaciones (ISP).	Separar interfaces o una clase base que encapsule la lista y `notificar()`.
`RecuerdoNivel` se protege con visibilidad de paquete.	Cualquier clase de `sokoban.modelo` podría leerlo.	Mover el originador y el recuerdo a un paquete propio.
La repetición se ejecuta sobre el juego real.	Depende de que el juego sea determinista.	Reproducir sobre una instancia separada de `JuegoSokoban`.
`PantallaJuego` arma la partida, maneja el teclado, crea comandos, controla la repetición y muestra carteles.	Varias responsabilidades (SRP).	Extraer la gestión de carteles y de la repetición.
`JuegoSokoban` guarda el motor como `MotorMovimiento`; la creación de cajas es un `switch` en `EstadoNivel`.	Dependencias de clases concretas.	Declarar el motor como `MediadorMovimiento`; Factory Method si crecen los tipos de caja.
No hay pruebas automatizadas.	Las reglas se verificaron jugando y con partidas simuladas.	Pruebas JUnit de `EstadoNivel`, `MotorMovimiento`, deshacer, DeLorean, repetición y puntaje.

9. Cómo ejecutar

Requisito: Java 17 o superior. El juego se ejecuta desde la **raíz del repositorio**, porque fuera del `.jar` las imágenes, los sonidos y los niveles se leen de las carpetas `assets/` y `niveles/`.

- **Con Maven:** `mvn package` y luego `java -jar target/sokoban.jar`. El `.jar` incluye las imágenes, los sonidos y los niveles.
- **Con un IDE** (VS Code, IntelliJ, Eclipse o NetBeans): abrir la carpeta, que se importa como proyecto Maven, y ejecutar `sokoban.Main`.
- **Sonidos (opcional):** `python docs/generar_sonidos.py` regenera los 16 archivos WAV.

Tecla o botón	Acción
Flechas	Mover al personaje
Botón Deshacer	Vuelve 5 movimientos atrás (dentro de los últimos 15, hasta 3 veces seguidas)
Botón Reiniciar	Reinicia el nivel
Esc o botón Pausa	Pausa (Continuar, Reiniciar, Menú)
T o botón DeLorean	Viaja al último punto en el tiempo (gasta una carga de plutonio)
M	Silencia o activa el sonido
«Ver repetición» (al terminar el nivel)	Reproduce la partida; Esc salta al final

10. Declaración de uso de herramientas de IA

La consigna permite usar herramientas de inteligencia artificial como apoyo al desarrollo, siempre que se declare en este informe. El equipo declara el siguiente uso.

Integrante	Herramienta	Uso
Martín Cabrera	Claude (Anthropic), asistente de IA	Integración de las partes: el enum `Epoca`, los cambios en `PanelTablero` y `PantallaJuego`, la unión de las ramas del repositorio y la recuperación de la documentación (Javadoc) que se había perdido al integrar el dominio. Código de las dos funcionalidades adicionales (DeLorean y repetición de la partida): lo escribió Claude a partir de la idea propuesta por el equipo (DeLorean) y de las indicaciones de Martín, y se probó con el juego en ejecución. **[Martín: confirmar o ajustar.]** Revisión del código contra la consigna. Guía explicativa del proyecto y diagramas UML. Presentación del avance. `README.md`. Redacción de este informe.
Luca de Iglesia	**[completar]**	**[Declarar si se usó IA en la interfaz gráfica y el contrato del modelo y, en ese caso, para qué.]**
Pedro Castillo y Lucía Santamarina	**[completar]**	**[Declarar si se usó IA en el dominio y los niveles y, en ese caso, para qué.]**
Mora Coghlan	**[completar]**	**[Declarar si se usó IA en el sonido y, en ese caso, para qué.]** Nota: `docs/sonido.md` ya indica que esta parte se desarrolló con asistencia de Claude (Anthropic); confirmar y detallar.

Responsabilidad. El equipo revisó, comprendió, validó y adaptó todo el material producido con asistencia de IA, y es responsable del código entregado y de las decisiones de diseño. El código se compiló con `javac --release 17` y con `mvn package`, y se probó en pantalla con los tres niveles. Cada integrante está en condiciones de explicar y defender cualquier parte del proyecto.

Referencias

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns* (3.ª ed.). Prentice Hall. Patrones GRASP.
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. Principios SOLID.
- Consigna del trabajo integrador, Proceso de Desarrollo de Software, Prof. Jonathan Pepe, UADE.